KNOWFLOW AI

Quy tắc sử dụng Git và GitHub

Hướng dẫn làm việc chung cho dự án DENSO Hackathon

Tài liệu này quy định cách cả nhóm làm việc trong cùng một repository, từ lúc nhận nhiệm vụ đến khi Pull Request được merge. Mục tiêu là giúp thành viên mới dùng Git có thể thao tác an toàn, giảm xung đột và giữ nhánh main luôn chạy được.

Kết luận chính: mỗi thành viên dùng branch riêng cho từng nhiệm vụ, không sửa trực tiếp main, không lưu dữ liệu lớn hoặc thông tin bí mật trong Git và phải tích hợp thường xuyên giữa hai đội.

Phiên bản 1.0
Ngày 09 tháng 09 năm 2026

Nội dung tài liệu

1 Các quy tắc bắt buộc

2 Vai trò và phạm vi sở hữu

3 Thiết lập lần đầu

4 Quy trình làm việc hằng ngày

5 Quy tắc đặt tên branch và commit

6 Pull Request và review

7 Quy tắc schema và dữ liệu

8 Xử lý xung đột

9 Các lỗi Git thường gặp

10 Các thao tác nguy hiểm

11 Điều kiện hoàn thành công việc

12 Bảng lệnh Git tra cứu nhanh

Cách sử dụng tài liệu

- Thành viên mới đọc mục 1, 3 và 4 trước khi nhận issue đầu tiên.
- Hai trưởng đội đọc thêm mục 2, 6 và 7 để quản lý tích hợp.
- Khi Git báo lỗi, dừng thao tác và tra mục 8 hoặc 9 trước khi thử lệnh khác.
- Nếu chưa chắc một lệnh có làm mất dữ liệu hay không, không chạy lệnh đó và hỏi repo maintainer.

1 Các quy tắc bắt buộc

Mười hai quy tắc dưới đây áp dụng cho mọi thành viên và mọi phần của dự án.

Số	Quy tắc	Lý do
1	Không push trực tiếp vào main	Mọi thay đổi phải qua Pull Request và review.
2	Mỗi issue dùng một branch riêng	Giới hạn phạm vi thay đổi và giúp quay lại dễ hơn.
3	Pull main trước khi bắt đầu	Giảm xung đột do làm việc trên mã nguồn cũ.
4	Commit nhỏ và có nội dung rõ	Người khác hiểu được thay đổi và dễ tìm lỗi.
5	Chỉ git add đúng file đã kiểm tra	Tránh commit nhầm file cá nhân hoặc dữ liệu lớn.
6	Không push force	Force push có thể xóa lịch sử của người khác.
7	Không commit .env hoặc API key	Git lưu lịch sử cả khi file đã bị xóa sau đó.
8	Không commit video và vector index	Các file này lớn, thay đổi thường xuyên và không phù hợp với Git.
9	Thay đổi schema phải có hai đội review	Schema là giao diện bàn giao giữa hai pipeline.
10	Không merge khi test đang lỗi	Main phải luôn ở trạng thái có thể chạy và demo.
11	Không tự xử lý xung đột bằng cách xóa bừa	Có thể làm mất thay đổi hợp lệ của đồng đội.
12	Không chia sẻ tài khoản GitHub	Mỗi người phải dùng tài khoản và lịch sử thao tác riêng.

Nguyên tắc khi chưa chắc chắn

Hãy chạy git status và git diff trước. Hai lệnh này chỉ đọc trạng thái và không làm mất dữ liệu. Nếu vấn đề vẫn chưa rõ, gửi ảnh lỗi cùng kết quả git status cho trưởng đội hoặc repo maintainer.

2 Vai trò và phạm vi sở hữu

Nhóm dùng một monorepo. Mỗi đội chịu trách nhiệm chính trong thư mục của mình, nhưng cùng dùng contracts, tests và evaluation.

Khu vực	Người phụ trách	Quy tắc review
contracts	Trưởng đội 1 và trưởng đội 2	Cả hai đội phải đồng ý trước khi merge.
acquisition	Đội 1	Ít nhất một thành viên đội 1 review.
retrieval	Đội 2	Ít nhất một thành viên đội 2 review.
apps	Người phụ trách giao diện	Đội sử dụng màn hình liên quan phải kiểm tra.
evaluation	Hai đội	Phải phân biệt dữ liệu synthetic và dữ liệu thật.
tests integration	Hai đội	PR làm thay đổi interface phải cập nhật test.

Repo maintainer

- Mời thành viên vào repository và phân quyền phù hợp.
- Bật bảo vệ branch main và cấm force push.
- Merge hoặc từ chối các PR ảnh hưởng đến nhiều module.
- Tạo tag cho các mốc demo và xử lý sự cố Git nghiêm trọng.

Trưởng đội

- Chia công việc thành issue có thể hoàn thành trong khoảng nửa ngày đến hai ngày.
- Theo dõi issue bị blocked và tổ chức integration test ít nhất hai lần mỗi tuần.
- Không tự đổi schema chỉ để thuận tiện cho riêng đội mình.

3 Thiết lập lần đầu

Phần việc của repo maintainer

Repository chỉ được khởi tạo một lần. Thành viên bình thường không chạy git init và không tự thêm remote origin trong một thư mục đã clone.

- Tạo repository denso-hackathon trên GitHub.
- Thêm README, .gitignore và commit đầu tiên trước khi mời cả nhóm làm việc.
- Mời từng tài khoản GitHub làm collaborator. Không dùng chung mật khẩu hoặc token.
- Bật yêu cầu Pull Request đối với main nếu quyền repository cho phép.

Phần việc của mỗi thành viên

Mở PowerShell trong thư mục muốn lưu dự án và chạy lần lượt các lệnh sau.

```
git config --global user.name "Ten cua ban"
git config --global user.email "email-github-cua-ban@example.com"
git clone https://github.com/duy32123/denso-hackathon.git
cd denso-hackathon
git remote -v
git branch --show-current
git status
```

Kết quả đúng là remote origin trỏ tới repository của nhóm, branch hiện tại là main và working tree không có thay đổi chưa commit.

Thiết lập file môi trường

Nếu dự án có .env.example, hãy sao chép thành .env trên máy cá nhân rồi điền khóa dịch vụ của riêng bạn. File .env phải nằm trong .gitignore và không được gửi qua commit, ảnh chụp màn hình hoặc nhóm chat công khai.

```
Copy-Item .env.example .env
git status
```

Kiểm tra trước khi nhận việc

- Chạy được lệnh git --version.
- Clone thành công repository thay vì tải file ZIP để lập trình hằng ngày.
- Đăng nhập đúng tài khoản GitHub đã được mời.
- Đọc README và chạy được project hoặc test cơ bản theo hướng dẫn.

4 Quy trình làm việc hằng ngày

Mỗi nhiệm vụ phải đi theo cùng một luồng từ issue đến Pull Request.

Bước 1 Nhận issue

- Đọc mục tiêu, đầu ra và điều kiện hoàn thành.
- Gán issue cho bản thân và chuyển trạng thái sang In progress.
- Nếu issue quá lớn hoặc chưa rõ đầu ra, hỏi trưởng đội trước khi code.

Bước 2 Cập nhật main và tạo branch

```
git switch main
git pull --ff-only origin main
git switch -c feature/42-video-catalog
```

Số 42 là số issue. Không tiếp tục code trên main sau lệnh pull.

Bước 3 Làm việc và tự kiểm tra

```
git status
git diff
# chạy test hoặc chạy chức năng vừa sửa
git add acquisition/video_catalog.py
git diff --cached
git commit -m "feat: add Haas video catalog loader"
```

Ưu tiên git add theo tên file. Chỉ dùng git add . sau khi đã đọc kỹ git status và chắc chắn không có file dữ liệu, .env hoặc file tạm.

Bước 4 Push và mở Pull Request

```
git push -u origin feature/42-video-catalog
```

- Mở Pull Request từ branch vừa push vào main.
- Liên kết issue bằng dòng Closes #42 trong phần mô tả.
- Ghi cách kiểm tra và ảnh chụp kết quả nếu thay đổi giao diện.
- Không tự merge khi chưa có review hoặc test đang lỗi.

Bước 5 Sau khi PR được merge

```
git switch main
git pull --ff-only origin main
git branch -d feature/42-video-catalog
```

5 Quy tắc đặt tên branch và commit

Tên branch

Loại	Mẫu	Ví dụ
Tính năng	feature/issue-noi-dung	feature/42-video-catalog
Sửa lỗi	fix/issue-noi-dung	fix/63-evidence-timestamp
Dữ liệu	data/issue-noi-dung	data/27-haas-metadata
Kiểm thử	test/issue-noi-dung	test/71-contract-validation
Tài liệu	docs/issue-noi-dung	docs/18-schema-guide
Bảo trì	chore/issue-noi-dung	chore/12-ci-config

- Dùng chữ thường, số và dấu gạch ngang.
- Tên ngắn nhưng phải thể hiện nội dung.
- Không dùng tên cá nhân như duy-branch hoặc test-moi.
- Không dùng branch dài hạn team-1 và team-2.

Nội dung commit

Dùng cấu trúc loại thay đổi, dấu hai chấm và mô tả ngắn bằng tiếng Anh.

Tiền tố	Dùng khi	Ví dụ
feat	Thêm chức năng	feat: add workflow importer
fix	Sửa lỗi	fix: preserve null rationale
test	Thêm hoặc sửa test	test: validate evidence ids
docs	Sửa tài liệu	docs: explain local setup
data	Sửa fixture hoặc metadata	data: add Haas video records
refactor	Đổi cấu trúc không đổi chức năng	refactor: split retrieval stages
chore	Cấu hình hoặc bảo trì	chore: update gitignore

Tránh các commit như update, fix bug, code mới hoặc final. Người review phải hiểu thay đổi mà không cần mở từng file.

6 Pull Request và review

Nội dung Pull Request

Mỗi Pull Request chỉ nên giải quyết một issue hoặc một thay đổi liên quan chặt chẽ.

Mục	Nội dung phải ghi
Mục tiêu	Issue nào được giải quyết và kết quả mong muốn.
Thay đổi	Các file hoặc module chính đã sửa.
Cách kiểm tra	Lệnh test hoặc các bước chạy thủ công.
Ảnh hưởng schema	Không có hoặc nêu rõ version thay đổi.
Dữ liệu	Có thay đổi fixture, metadata hoặc index hay không.
Rủi ro	Điểm chưa chắc chắn hoặc trường hợp chưa kiểm tra.

Checklist của người mở PR

- Đã đọc lại git diff và danh sách file thay đổi.
- Đã chạy test liên quan và ghi kết quả.
- Không chứa .env, API key, video, database hoặc file sinh tự động.
- Không có log debug, code bỏ dở hoặc file cá nhân.
- Đã cập nhật tài liệu nếu cách sử dụng thay đổi.
- Đã cập nhật ví dụ và contract test nếu schema thay đổi.

Checklist của người review

- Thay đổi có đúng issue và không mở rộng phạm vi không cần thiết hay không.
- Có làm hỏng module của đội còn lại hoặc interface chung không.
- Test có kiểm tra trường hợp lỗi và trường null hay chỉ kiểm tra ca đẹp.
- Dữ liệu thật và synthetic có được phân biệt rõ không.
- Hướng dẫn chạy có đủ để một người khác kiểm tra lại không.

Review tập trung vào code và tác động kỹ thuật. Ghi rõ cần sửa gì và vì sao; không chỉ viết sai, chưa ổn hoặc làm lại.

7 Quy tắc schema và dữ liệu

Thay đổi schema

Schema là hợp đồng giữa đội 1 và đội 2. Một thay đổi nhỏ đối với tên field, kiểu dữ liệu hoặc ý nghĩa của null đều có thể làm importer và retrieval bị lỗi.

1. Tạo issue có nhãn shared và mô tả lý do thay đổi.
2. Cập nhật JSON Schema, ví dụ real, ví dụ synthetic và contract test trong cùng PR.
3. Cập nhật schema_version và CHANGELOG.
4. Hai trưởng đội hoặc hai người được ủy quyền cùng review.
5. Không đổi schema âm thầm trong code riêng của một đội.

Dữ liệu được phép lưu trong Git

Được lưu	Không được lưu
Schema và tài liệu	API key, token và mật khẩu
Metadata và URL nguồn được phép chia sẻ	Video, audio và keyframe số lượng lớn
Fixture nhỏ phục vụ test	Vector database và embedding cache
Synthetic samples có gắn nhãn	Model weights và file sinh tự động lớn
Evaluation set được phép phân phối	Dữ liệu nội bộ DENSO khi chưa được phép

Danh sách gitignore tối thiểu

```
.env
.venv/
__pycache__/
*.mp4
*.wav
*.mp3
data/raw/
data/processed/
vector_store/
*.db
```

Một file đã commit trước khi thêm vào .gitignore vẫn tiếp tục được Git theo dõi. Khi gặp trường hợp này, báo repo maintainer để gỡ file khỏi lịch sử theo cách phù hợp; không chỉ xóa file rồi coi như bí mật đã an toàn.

8 Xử lý xung đột

Xung đột xảy ra khi hai thay đổi chạm vào cùng một phần của file và Git không thể tự quyết định nội dung nào đúng. Không chọn Accept Current hoặc Accept Incoming chỉ vì nút đó nhanh hơn.

Cập nhật branch bằng main

```
git status
git fetch origin
git switch feature/42-video-catalog
git merge origin/main
```

Nếu Git báo conflict, mở từng file được liệt kê trong git status. Git đánh dấu vùng xung đột như sau.

```
<<<<<<< HEAD
noi dung tren branch cua ban
=======
noi dung tu main
>>>>>>> origin/main
```

1. Đọc cả hai phần và xác định logic cuối cùng cần giữ.
2. Trao đổi với tác giả của phần code còn lại nếu ý nghĩa chưa rõ.
3. Xóa các dấu đánh dấu conflict sau khi đã ghép nội dung đúng.
4. Chạy lại test rồi mới add và commit.

```
git add path/to/conflicted_file.py
git commit -m "merge: resolve conflict with main"
git push
```

Hủy lần merge đang làm

Nếu chưa sửa xong và muốn quay lại trạng thái trước khi merge, dùng lệnh sau.

```
git merge --abort
```

Nếu lệnh abort không chạy hoặc bạn đã dùng thêm nhiều lệnh khác, dừng lại và nhờ repo maintainer. Không dùng reset hard hoặc force push để tự cứu khi chưa hiểu trạng thái repository.

9 Các lỗi Git thường gặp

Lỗi dubious ownership

Lỗi này xuất hiện khi Windows cho biết thư mục repository thuộc một tài khoản khác. Chỉ đánh dấu safe directory khi bạn biết chính xác thư mục đó và tin cậy nội dung bên trong.

```
git config --global --add safe.directory "C:/duong-dan-chinh-xac/toi/denso-hackathon"
```

Không đặt safe.directory thành dấu sao cho mọi thư mục. Nếu repository đến từ nguồn không rõ hoặc máy dùng chung, hãy hỏi người quản lý máy trước.

Lỗi src refspec main does not match any

Nguyên nhân thường là repository chưa có commit hoặc branch hiện tại không tên main. Thành viên bình thường nên clone repository đã được repo maintainer khởi tạo. Chỉ maintainer xử lý repository mới bằng chuỗi lệnh sau.

```
git status
git branch --show-current
git add README.md .gitignore
git commit -m "chore: initialize repository"
git branch -M main
git push -u origin main
```

Lỗi remote origin already exists

```
git remote -v
git remote set-url origin https://github.com/duy32123/denso-hackathon.git
```

Không thêm origin lần thứ hai. Hãy kiểm tra URL hiện tại rồi mới thay đổi.

Lỗi push bị rejected hoặc non fast forward

Không dùng force push. Cập nhật thay đổi từ main vào branch của bạn rồi giải quyết conflict nếu có.

```
git fetch origin
git switch ten-branch-cua-ban
git merge origin/main
git push
```

Lỗi authentication failed

- Kiểm tra bạn đã đăng nhập đúng tài khoản được mời vào repository.
- Đăng nhập qua trình duyệt hoặc phương thức xác thực GitHub được máy hỗ trợ.
- Không gửi token, mã đăng nhập hoặc ảnh chứa thông tin xác thực cho người khác.

10 Các thao tác nguy hiểm

Các lệnh dưới đây có thể xóa thay đổi, xóa file chưa commit hoặc viết lại lịch sử. Thành viên mới không chạy nếu chưa có hướng dẫn trực tiếp từ repo maintainer.

Lệnh hoặc hành động	Rủi ro
git push --force	Ghi đè lịch sử trên GitHub và có thể làm mất commit của người khác.
git reset --hard	Xóa thay đổi chưa commit trong working tree và index.
git clean -fd	Xóa file và thư mục chưa được Git theo dõi.
git restore .	Bỏ toàn bộ thay đổi chưa commit trong phạm vi hiện tại.
Xóa thư mục .git	Làm mất metadata, branch và lịch sử Git cục bộ.
Commit file .env	Có thể làm lộ khóa dịch vụ ngay cả khi file được xóa ở commit sau.

Trước mọi thao tác khôi phục, xác định chính xác file hoặc commit cần xử lý. Ưu tiên phương án có thể kiểm tra và hoàn tác, chẳng hạn tạo commit sửa lỗi hoặc dùng revert theo hướng dẫn của maintainer.

11 Điều kiện hoàn thành công việc

Một issue chỉ chuyển sang Done khi đáp ứng đầy đủ các điều kiện sau.

- Code đã merge vào main thông qua Pull Request.
- Test liên quan chạy thành công trên máy người làm và trên CI nếu có.
- Một thành viên khác có thể chạy lại theo hướng dẫn.
- Không chứa secret, dữ liệu lớn, log debug hoặc file cá nhân.
- Schema, ví dụ và tài liệu đã được cập nhật nếu interface thay đổi.
- Issue được liên kết với PR và đóng sau khi merge.
- Main vẫn chạy được luồng tích hợp tối thiểu của dự án.

Kiểm tra tích hợp tối thiểu

Cuối mỗi tuần, nhóm phải kiểm tra ít nhất một workflow thật đi hết chuỗi import knowledge base, đặt câu hỏi, truy xuất claim, sinh câu trả lời và mở đúng evidence. Kết quả trên synthetic KB không thay thế được phép kiểm tra này.

12 Bảng lệnh Git tra cứu nhanh

Mục đích	Lệnh
Xem trạng thái	git status
Xem thay đổi chưa add	git diff
Xem thay đổi đã add	git diff --cached
Xem branch hiện tại	git branch --show-current
Xem remote	git remote -v
Chuyển sang main	git switch main
Cập nhật main	git pull --ff-only origin main
Tạo branch	git switch -c feature/42-ten-nhiem-vu
Add một file	git add path/to/file
Commit	git commit -m "feat: mo ta thay doi"
Push lần đầu	git push -u origin ten-branch
Push các lần sau	git push
Lấy thông tin mới	git fetch origin
Ghép main vào branch	git merge origin/main
Hủy merge đang làm	git merge --abort
Xóa branch đã merge	git branch -d ten-branch

Trình tự an toàn cần nhớ

```
git switch main
git pull --ff-only origin main
git switch -c feature/issue-ten-nhiem-vu
# sua code va chay test
git status
git diff
git add path/to/file
git diff --cached
git commit -m "feat: mo ta thay doi"
git push -u origin feature/issue-ten-nhiem-vu
# mo Pull Request tren GitHub
```

Khi gặp lỗi không có trong tài liệu, gửi ba thông tin cho người hỗ trợ: lệnh vừa chạy, toàn bộ thông báo lỗi và kết quả git status. Không gửi API key, token hoặc nội dung file .env.